

Proyecto Fase Final

Rocío Anahí, Quinto Reyes, 202500234

Rodrigo Emmanuel Díaz García, 202503397

John Eleazar Carias Orozco, 202505240

*Escuela de Mecánica Eléctrica, Facultad de Ingeniería,
Universidad de San Carlos de Guatemala*

Esta actividad forma parte del primer proyecto del curso de INTRODUCCIÓN A LA PROGRAMACIÓN DE COMPUTADORAS (0769) y consistió en el desarrollo de la fase final de un sistema digital de control de entrada y salida de vehículos utilizando el lenguaje de programación Python. El objetivo principal fue implementar la lógica del sistema, aplicando conceptos como programación orientada a objetos, el uso de estructuras de control y el manejo de datos. El sistema permite registrar la placa de los vehículos, almacenar la hora de entrada y gestionar la información necesaria para el control del parqueo. Además, se trabajó con funciones y clases para organizar mejor el código y facilitar su comprensión. Otro punto importante fue el manejo de archivos de texto, lo cual permitió guardar y leer información para que los datos no se pierdan al cerrar el programa. Finalmente, esta práctica ayudó a reforzar el uso del entorno de desarrollo, la ejecución del programa y la correcta organización del proyecto para su entrega y respaldo en GitHub.

A. Objetivos

- Refinar la lógica del control de vehículos para asegurar la integridad de los datos, implementando validaciones estrictas para formatos de placa y cálculos precisos de tiempo de estancia.
- Completar el desarrollo visual y funcional del sistema, logrando una navegación intuitiva que facilite el registro de salida y la consulta de historial de manera fluida.
- Asegurar que el sistema no solo recupere información, sino que genere registros detallados o reportes finales de ocupación, garantizando que la solución sea técnica y operativamente completa.

B. Marco Teórico

Lenguaje de Programación: Python

Para el desarrollo de este proyecto utilizamos Python, un lenguaje que hemos estudiado durante un tiempo en nuestra clase. Este se destaca por su versatilidad y su sintaxis clara, que nos permite programar de manera más fluida y sencilla, a comparación de otros lenguajes como C. Python nos facilita la estructura inicial y el mantenimiento a largo plazo en aplicaciones sencillas o proyectos más complejos

Programación Orientada a Objetos (POO)

La estructura de nuestro sistema se fundamenta en el concepto aprendido de Programación Orientada a Objetos. Mediante el uso de clases y objetos, se logró abstraer datos de los vehículos para poder obtener sus componentes digitales según los respectivos registros. Este enfoque ayuda a organizar mejor la lógica y permite la reutilización de código.

Librería

datetime

El manejo de datos temporales es gestionado a través de la librería nativa datetime. Su integración es vital para el núcleo del programa, ya que permite capturar con precisión las marcas temporales de entrada y realizar el cálculo automático del tiempo de permanencia de cada unidad dentro del sistema.

Librería

os

Para la interacción directa con el sistema operativo, se implementó la librería os. Esta herramienta es la encargada de administrar el sistema de archivos, permitiendo que el programa verifique la existencia de rutas, cree directorios de almacenamiento y gestione los recursos del equipo de manera automática.

Manejo de Archivos de Texto

Con el fin de garantizar la persistencia de los datos, se optó por el uso de archivos planos (.txt). Este método permite volcar la información procesada en memoria hacia un soporte físico, asegurando que los registros de los vehículos se conserven intactos, incluso después de cerrar la aplicación.

Librería

tkinter

Para la interfaz gráfica de usuario, se ha considerado el uso de tkinter. Aunque el desarrollo actual se ha centrado en el funcionamiento, la lógica interna y el procesamiento de datos, esta librería servirá como puente en fases futuras para ofrecer una experiencia visual más agradable y accesible para los usuarios finales.

Entorno de Desarrollo y Control de Versiones

La escritura y depuración del código se llevó a cabo en un entorno de desarrollo integrado (IDE), que optimizó el flujo de trabajo. Al mismo tiempo, se utilizó GitHub como eje central para el control de versiones, lo que permitió gestionar el historial de cambios, mantener respaldos constantes en la nube y coordinar las contribuciones de cada integrante del equipo, de forma organizada.

C. Marco Practico

El proceso para la elaboración de este proyecto consistió en la preparación lógica por parte de los integrantes para ordenar las ideas de como se iba a realizar el proyecto; luego se procedió a escribir el código, el cual se dividió en varios archivos y carpetas para que todo quedara ordenado y fácil de encontrar. La carpeta principal está formada por un archivo principal llamado “main.py”, luego de una subcarpeta donde se organizan los datos de usuarios, bitácoras, vehículos, etcétera, en archivos “.txt”; y de otra subcarpeta donde se guarda el resto del código. Es aquí donde tenemos los archivos de código; cuatro de ellos se usaron para crear clases donde se guardan las funcionalidades básicas del sistema de parqueo, otro que es un sistema que maneja las interacciones entre el usuario, los archivos de texto y todo el proceso lógico, y dos archivos donde se guarda la interfaz grafica hecha con Tkinter y la opción de ejecutar el programa en una consola de comandos.

D. Código

```

769-Proyecto-1 > src > controlador > sistema.py
1 import os
2 from datetime import datetime
3 from src.modelo.usuario import Usuario
4 from src.modelo.vehiculo import Vehiculo
5 from src.modelo.movimiento import Movimiento
6 from src.modelo.parking import Parking
7
8
9 class Sistema:
10     def __init__(self):
11         self.parking = Parking()
12         self.usuario_actual = None
13         self.crear_estructura()
14         self.inicializar_config()
15
16     # -----
17     # ESTRUCTURA
18     # -----
19     def crear_estructura(self):
20         rutas = [
21             "data/configuracion",
22             "data/vehiculos",
23             "data/movimientos",
24             "data/reportes",
25             "data/auditoria"
26         ]
27         for ruta in rutas:
28             os.makedirs(ruta, exist_ok=True)

```

```

769-Proyecto-1 > src > controlador > sistema.py
30 def inicializar_config(self):
31     ruta = "data/configuracion/tarifa.txt"
32     if not os.path.exists(ruta):
33         with open(ruta, "w") as f:
34             f.write("tarifa_horas\n")
35
36     # -----
37     # TARIFA
38     # -----
39     def obtener_tarifa(self):
40         with open("data/configuracion/tarifa.txt", "r") as f:
41             return float(f.readline().split("\n")[1])
42
43     def actualizar_tarifa(self, nueva):
44         with open("data/configuracion/tarifa.txt", "w") as f:
45             f.write(f"tarifa_horas({nueva})\n")
46
47     # -----
48     # BITACORA
49     # -----
50     def log(self, mensaje):
51         with open("data/auditoria/bitacora.txt", "a") as f:
52             hora = datetime.now().strftime("%H:%M")
53             usuario = self.usuario_actual if self.usuario_actual else "Sistema"
54             f.write(f"({hora}) {usuario} -> {mensaje}\n")
55
56     # -----
57     # USUARIOS
58     # -----

```

```

769-Proyecto-1 > src > controlador > sistema.py
59 def registrar_usuario(self, username, password, rol):
60     ruta = "data/configuracion/usuarios.txt"
61
62     if os.path.exists(ruta):
63         with open(ruta, "r") as f:
64             for linea in f:
65                 if linea.split(",")[0] == username:
66                     return "Usuario ya existe"
67
68     with open(ruta, "a") as f:
69         f.write(Usuario(username, password, rol).to_txt())
70
71     self.log(f"Usuario creado: {username}")
72     return "Usuario registrado"
73
74 def login(self, username, password):
75     ruta = "data/configuracion/usuarios.txt"
76
77     if not os.path.exists(ruta):
78         return False, None
79
80     with open(ruta, "r") as f:
81         for linea in f:
82             u, p, r = linea.strip().split(",")
83             if u == username and p == password:
84                 self.usuario_actual = u
85                 self.log("Inicio de sesion")
86                 return True, r
87     return False, None

```

```

769-Proyecto-1 > src > controlador > sistema.py
88 def obtener_usuarios(self):
89     ruta = "data/configuracion/usuarios.txt"
90     if not os.path.exists(ruta):
91         return []
92     with open(ruta, "r") as f:
93         return f.readlines()
94
95 # -----
96 # VEHICULOS
97 # -----
98 def registrar_vehiculo(self, placa, tipo):
99     try:
100         v = Vehiculo(placa, tipo)
101         ruta = f"data/vehiculos/{v.placa}.txt"
102
103         if os.path.exists(ruta):
104             return "Vehiculo ya existe"
105
106         with open(ruta, "w") as f:
107             f.write(v.to_txt())
108
109         self.log(f"Registro vehiculo: {v.placa}")
110         return "Vehiculo registrado"
111     except ValueError as e:
112         return str(e)
113
114 def obtener_vehiculos(self):

```

```

769-Proyecto-1 > src > controlador > sistema.py
117     return os.listdir("data/vehiculos")
118
119 # -----
120 # MOVIMIENTOS
121 # -----
122 def registrar_entrada(self, placa):
123     placa = placa.upper()
124     ruta_vehiculo = f"data/vehiculos/{placa}.txt"
125
126     if not os.path.exists(ruta_vehiculo):
127         return "Vehiculo no registrado"
128
129     if placa in self.parking.ocupados:
130         return "El vehiculo ya está dentro"
131
132     if not self.parking.hay_espacio():
133         return "Parqueo lleno"
134
135     if placa not in self.parking.ocupados:
136         self.parking.ocupados.append(placa)
137
138     mov = Movimiento(placa)
139     ruta = f"data/movimientos/{placa}_historial.txt"
140
141     with open(ruta, "a") as f:
142         f.write(mov.to_txt_entrada())
143
144     self.log(f"Entrada: {placa}")
145     return f"Entrada registrada: {placa}"

```

```

769-Proyecto-1 > src > controlador > sistema.py
146
147 def registrar_salida(self, placa):
148     placa = placa.upper()
149
150     if placa not in self.parking.ocupados:
151         return f"{placa} no está en el parqueo"
152
153     tarifa = self.obtener_tarifa()
154     mov = Movimiento(placa)
155     mov.salida = datetime.now()
156     mov.total = tarifa
157
158     ruta = f"data/movimientos/{placa}_historial.txt"
159
160     with open(ruta, "a") as f:
161         f.write(mov.to_txt_salida())
162
163     self.parking.retirar(placa)
164
165     self.log(f"Salida: {placa} - Q:{mov.total}")
166     return f"Salida registrada. Total a pagar: Q{mov.total}"
167
168 def vehiculos_activos(self):
169     return [v for v in self.parking.ocupados if v is not None]
170
171 # -----
172 # REPORTES
173 # -----
174

```

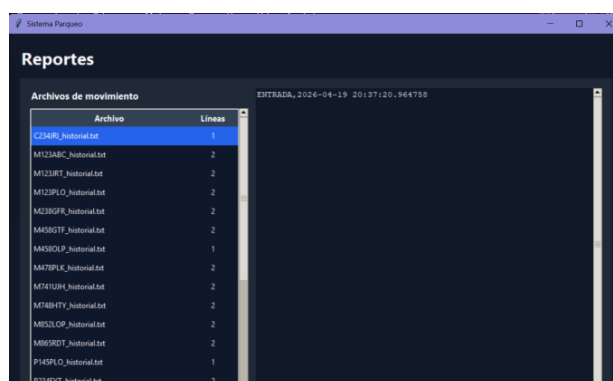
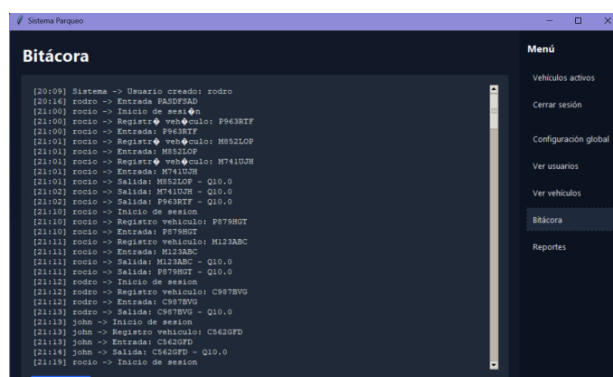
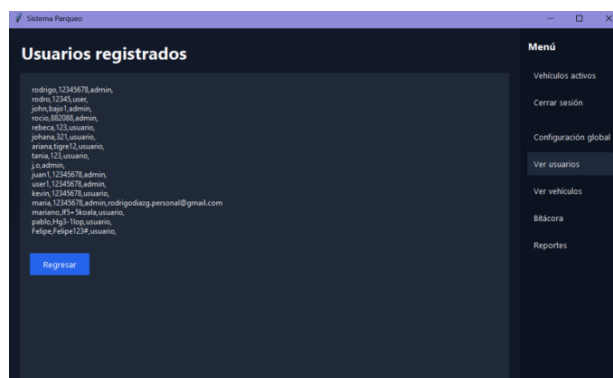
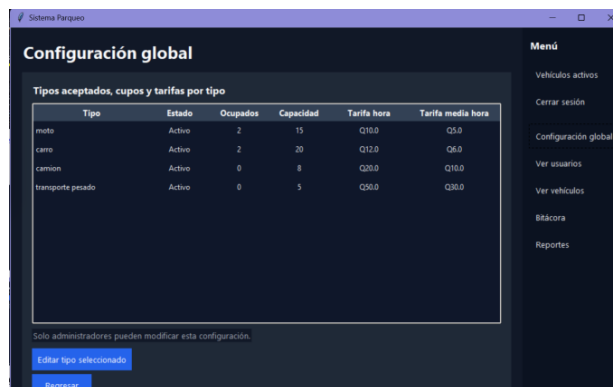
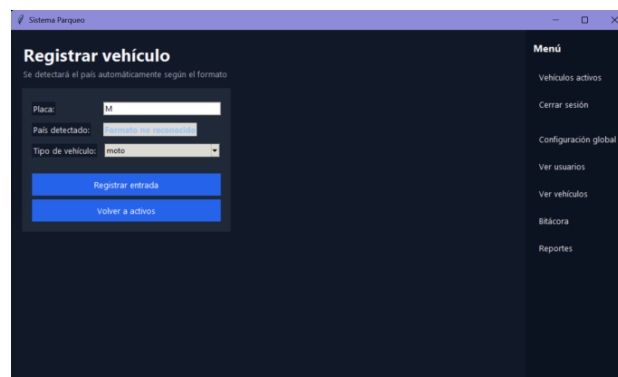
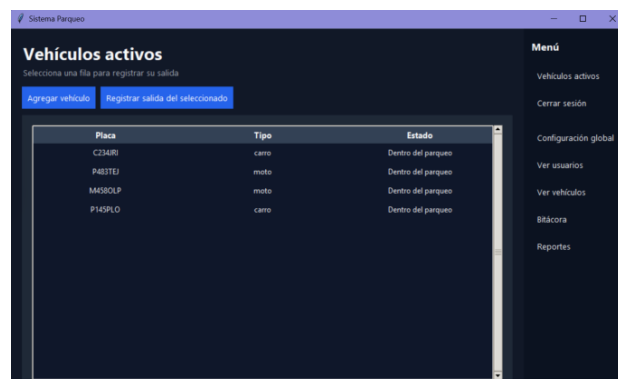
```

769-Proyecto-1 > src > controlador > sistema.py
172 # -----
173 # REPORTES
174 # -----
175 def reporte_movimientos(self):
176     carpeta = "data/movimientos"
177     datos = []
178
179     for archivo in os.listdir(carpeta):
180         with open(os.path.join(carpeta, archivo)) as f:
181             datos.append((archivo, len(f.readlines())))
182
183     return datos
184
185 def ver_bitacora(self):
186     ruta = "data/auditoria/bitacora.txt"
187     if not os.path.exists(ruta):
188         return []
189     with open(ruta) as f:
190         return f.readlines()

```

<https://github.com/rodrigoedg0/769-Proyecto>

E. Resultados



F. Conclusiones

- Se logró consolidar un sistema de control digital íntegro, donde la lógica de entrada, procesamiento y salida de vehículos funciona de manera sincronizada y sin errores de ejecución.
- La integración final de las sugerencias del auxiliar permitió transformar las correcciones iniciales en una estructura de código sólida, eficiente y adaptable a futuras mejoras.
- El final del proyecto permitió al equipo dominar el uso de herramientas de desarrollo y mejorar el trabajo en equipo, logrando la calidad final deseada.

G. Referencias

Python Software Foundation. (s. f.). tkinter
Python interface to Tcl/Tk.
<https://docs.python.org/3/library/tkinter.html>

Python Software Foundation. (s. f.). datetime
Basic date and time types.
<https://docs.python.org/3/library/datetime.html>

Python Software Foundation. (s. f.). os
Miscellaneous operating system interfaces.
<https://docs.python.org/3/library/os.html>

